Praeses Technical Sample

Task

You will build a Blackjack game as a web application using Ruby on Rails or a comparable modern full-stack framework. This project is an opportunity to demonstrate the caliber of your work by showcasing your ability to design and structure a functioning web application. The focus should be on the application's architecture and game logic rather than on visual design. A straightforward interface is acceptable, but the app should clearly demonstrate how you organize game logic, manage state, and structure the project.

Important Note on AI Usage: This assignment is intended to evaluate your own engineering ability and familiarity with modern web development. The expectation is that you understand and can produce the necessary code yourself. Submissions should reflect your own design decisions, problem-solving, and coding practices. **Because this role involves working on DoD projects where external AI-assisted coding tools (e.g., ChatGPT, GitHub Copilot, Claude) are not permitted, we ask that you complete this assignment without using such tools.**

Evaluation

Adherence to Game Rules (High)

Like most card games, Blackjack is heavily based on rules. As part of this task, you will need to identify rules for the core game and implement those in your submission. Bicycle Cards has a good set of rules. But please consider betting, splitting, and multi-player non-essential features. They may be included if you wish, but they are less important than the core of the game. The most important factors here are: "dealing" cards in the correct order, identifying win conditions, handling player actions appropriately, handling the value of aces.

Overall Impression (Medium)

A holistic impression of the submission. This includes, but is not limited to, completeness, polish, and how well the solution demonstrates your approach to problem-solving and coding practices.

Program Structure and Design (Medium-High)

This covers how well designed the application is. Is the code broken up into multiple files, classes, functions, etc? Is each piece of the application focused and cohesive?

Documentation and Readability (Medium)

This focuses on how easy the submission is to read and understand. Ensure that variables and functions are appropriately named. Code comments are not required, but feel free to add if needed to enhance clarity.

README(Required)

A short README should be included as part of your submission (a simple README.md file in the repository will suffice). It should indicate any additional features that you have implemented beyond the core game rules. Additionally, it should include instructions on how to build and run your application.

UI / Presentation (Low-Med)

The user interface for your Blackjack web application does not need to be highly polished or visually complex. A simple, functional presentation is sufficient. The primary expectation is that the application runs as a web app and allows a user

to play the game through a web interface. Focus on clarity and usability rather than advanced styling. What matters most is how the UI demonstrates your integration of game logic and state management

Additional Features (Low)

Additional consideration will be given for any features that are implemented beyond the core game, but these are considered supplementary to the core requirements and are not, themselves, required to complete this task. Some examples include multiplayer, betting, and splitting. These are just a few examples of features that you *could* add.

Live Review

After you submit your assignment, our engineering team will review your code. If your submission aligns well with what we're looking for, we'll invite you to a live code review session. During this session, you'll share your screen and walk us through your implementation, including stepping through the full flow of a Blackjack hand so we can understand how your game logic behaves end-to-end. You will also be asked to make changes to your code in real time—this may include fixing a bug, adding a small feature, or adjusting part of your game flow. Some changes will be simple enough to implement on the spot; others may be complex enough that we'll ask you to talk through how you would approach them. The purpose of this session is to confirm your technical competence: your ability to understand your own code, reason about it, and make or explain changes effectively.